FACULDADE SENAI ARMANDO DE ARRUDA PEREIRA

MARCIO PETIGROSSO

RA 25178622

TESTES DE PENETRAÇÃO E ETHICAL HACKING

Relatório técnico de teste de injeção SQL em formulário de autenticação

São Caetano do Sul
2026

Resumo

Este relatório técnico registra o teste de injeção SQL realizado no formulário de autenticação de uma aplicação PHP de treinamento. O procedimento executado seguiu o tutorial de Segurança Cibernética do SENAI: instalação de Notepad++ e XAMPP, obtenção dos arquivos de treinamento, configuração do servidor local, importação de sqltraining.sql, cadastro de usuário, testes de autenticação, inserção de duas entradas de ataque e alteração de login1.php. A execução mostrou que a concatenação de entradas ao SQL modifica a condição de autenticação. A defesa indicada no tutorial, baseada em addslashes(), foi aplicada e discutida, com reteste das credenciais válidas e das mesmas entradas de ataque. Como complemento, apresenta-se uma alternativa com consulta parametrizada. Conclui-se que separar dados e instruções SQL é a medida central para corrigir a causa da vulnerabilidade. Os resultados obtidos no laboratório são coerentes com a análise do código lido em login1.php.

1 Introdução e objetivo

O objetivo é explicar a técnica de invasão e a defesa aplicadas ao teste realizado na atividade, documentando cada um dos cinco critérios de avaliação. O objeto de análise é o formulário login1.php da aplicação de treinamento, com consulta à tabela users. O ataque ficou delimitado ao contorno da autenticação em laboratório local. A execução não envolveu sistema de terceiros e não incluiu extração ou alteração de dados.

A metodologia seguiu as etapas do tutorial, executadas no ambiente de treinamento: preparação do ambiente, configuração da aplicação e do banco, verificação da autenticação, análise das entradas de ataque, alteração do código e reteste. Cada etapa apresenta o procedimento, a finalidade e o resultado obtido, de modo a acompanhar a relação entre a vulnerabilidade, a defesa e as evidências.

A interpretação do teste confronta três registros do mesmo fato. O primeiro é o comportamento da aplicação nas oito verificações da seção 4. O segundo é a consulta concatenada lida em login1.php. O terceiro são as orientações de correção, que divergem entre si: o tutorial encaminha addslashes(), o manual do PHP adverte que essa função não constitui proteção específica para consultas SQL, e a OWASP indica consultas parametrizadas. O relatório pondera essas posições com o que o código e os testes sustentam.

2 Fundamentos da técnica

Injeção SQL ocorre quando dados externos interferem na estrutura de uma instrução enviada ao banco de dados. Neste caso, o conteúdo do campo de usuário é concatenado a uma consulta SELECT. Quando esse conteúdo introduz aspas, uma condição lógica e um comentário SQL, a consulta deixa de exigir a combinação correta de usuário e senha. A

vulnerabilidade está na construção da consulta; não depende de descobrir ou quebrar o hash da senha.

O teste de penetração executado reproduziu a exploração para verificar a falha e orientar a correção. Seu caráter ético dependeu de autorização e de escopo definido. A aplicação de treinamento foi preparada em localhost. O impacto observado foi o acesso indevido à área autenticada.

O impacto observado corresponde ao ingresso na área autenticada com senha incorreta, produzido pela alteração da consulta. A conclusão fica restrita a essa falha de identidade da aplicação, que é o escopo em que o teste foi executado.

3 Procedimentos do teste

3.1 Instalação dos softwares necessários

O teste utilizou o ambiente Windows descrito no tutorial. No Notepad++, o instalador foi obtido e executado, com seleção de Português Brasileiro, avanço no assistente, aceitação da licença, manutenção das opções do exercício, criação do atalho e conclusão da instalação. O editor foi aberto para verificação e utilizado na edição de login1.php. O resultado obtido foi um editor disponível para leitura e edição dos arquivos PHP.

Para o XAMPP, a edição para Windows foi obtida no site Apache Friends, o instalador foi executado, as opções apresentadas foram avançadas, o idioma foi selecionado e a instalação foi concluída. O ambiente ficou na estrutura C:\xampp. A inicialização dos serviços usou os botões Start de Apache e MySQL no painel de controle, com os indicadores de serviço iniciado. Apache atendeu as páginas e executou PHP; o serviço de banco atendeu às consultas da aplicação.

As permissões de rede solicitadas pelo sistema operacional devem respeitar o escopo local do laboratório. A aplicação vulnerável permaneceu restrita ao ambiente de treinamento.

3.2 Obtenção e organização dos arquivos

A página 11 do tutorial indica um pacote hospedado no Google Drive. O pacote foi baixado, salvo no computador e extraído pela opção Extrair tudo. A organização final ficou com a pasta www dentro de C:\xampp\htdocs, sem duplicação de diretórios, e a página inicial respondeu em http://localhost/www/.

Os componentes identificados no material são sqltraining.sql, utilizado na importação do banco, register.php, utilizado no cadastro, e login1.php, utilizado no teste de autenticação e na correção. O arquivo SQL foi localizado no conteúdo extraído antes da importação.

A conferência do caminho integra a validade do teste. A página retestada é a de C:\xampp\htdocs\www\login1.php, a mesma alterada no editor. Um diretório www aninhado dissociaria o arquivo editado do endereço aberto no navegador, e a comparação entre código e comportamento perderia correspondência.

3.3 Configuração do servidor e do banco de dados

Com os serviços ativos, a administração do banco utilizou o endereço http://localhost/phpmyadmin no navegador. A importação foi feita em Import, com o arquivo sqltraining.sql selecionado em Choose File e confirmado em Go. A importação foi concluída sem erros, e a estrutura de treinamento ficou disponível, incluindo a tabela users referenciada no código.

A validação da configuração utilizou http://localhost/www/. O carregamento da página inicial verificou o atendimento da aplicação; o cadastro de uma conta e o login verificaram, adicionalmente, a integração com o banco. A abertura da página inicial, isoladamente, não comprova a conexão com o banco. A configuração dos dados concentrou-se na importação de sqltraining.sql, seguida da verificação funcional pelo formulário.

Embora a aplicação seja acessada localmente, os registros cadastrados podem persistir no banco instalado no computador. O uso de localhost não significa ausência de armazenamento nem garante, por si só, que os serviços não estejam expostos à rede. O teste permaneceu restrito ao laboratório e utilizou exclusivamente dados fictícios.

3.4 Cadastro e verificação inicial da autenticação

O cadastro começou pela seleção de register.php — user registration page na página inicial. Para manter os testes comparáveis, adotou-se o usuário fictício aluno_teste e a senha fictícia Laboratorio123. O preenchimento dos demais campos exigidos pelo formulário utilizou dados de teste. Após Enviar, a aplicação ingressou automaticamente na área autenticada, apresentada como um cadastro de itens de supermercado inicialmente sem produtos.

A etapa seguinte consistiu em acionar Logout e selecionar login1.php — basic injection page. Com aluno_teste e Laboratorio123, o acesso foi permitido. Após novo logout, aluno_teste com SenhaIncorreta teve o acesso rejeitado. Esses controles estabeleceram a linha de base da autenticação antes da exploração. O retorno por Home permitiu selecionar novamente a página de login.

A linha de base torna interpretáveis as tentativas seguintes. Com o nome de usuário literal, a senha correta foi aceita e a senha incorreta foi recusada. O formulário, antes da injeção, distinguia credencial válida de credencial inválida. O acesso obtido depois, ainda com SenhaIncorreta, fica associado à entrada que altera a consulta.

3.5 Inserção dos comandos de teste

O tutorial permite usar um dos dois comandos; este relatório inclui ambos para documentar integralmente as entradas apresentadas. Cada comando foi inserido isoladamente no campo Usuário de login1.php e submetido pelo botão Enviar. Nesses testes, utilizou-se SenhaIncorreta no campo de senha, para evidenciar que o acesso obtido não dependeu da credencial correta. Houve logout entre as tentativas com acesso permitido.

Entrada A

```
asd' or 1=1 -- //
```

Entrada B

```
- ' OR 1=1 -- //
```

No código vulnerável, as duas entradas resultaram em acesso à área autenticada, com SenhaIncorreta no campo de senha. A aplicação tratou o retorno de registros como autenticação bem-sucedida. Esse efeito coincide com a consulta reconstruída na seção 3.6 a partir do código lido e da entrada submetida. A resposta visível foi o ingresso na área autenticada. A conta selecionada e a quantidade de registros devolvidos não constam dessa resposta; por isso o relatório não atribui a sessão a um usuário específico.

3.6 Análise do código vulnerável

A consulta apresentada na página 21 concatena o nome de usuário recebido e o hash MD5 da senha. A quebra de linha abaixo apenas facilita a leitura do mesmo comando PHP:

```
$q = "SELECT * FROM users where username='" . $username
. "' AND password = '" . md5($pass) . "'";
```

Com a entrada A, a consulta resultante da concatenação assume a forma abaixo. As reticências representam o hash calculado para a senha, que permanece no trecho comentado:

```
SELECT * FROM users where username='asd' or 1=1 -- //' AND password='...'
```

Na consulta, o trecho posterior a -- fica comentado. A aspa após asd encerra o valor de username; OR 1=1 introduz uma condição sempre verdadeira; -- seguido de espaço inicia o comentário no contexto do banco do exercício. Os caracteres // são parte desse comentário, não seu iniciador. Assim, a comparação da senha deixa de restringir a seleção.

Na entrada B, o texto anterior à aspa é diferente, mas a lógica é a mesma. A tautologia OR 1=1 torna a condição verdadeira para linhas existentes. A falha de autenticação decorre de o programa aceitar esse retorno como prova de identidade. O código lido e a resposta da aplicação deixam em aberto qual conta foi selecionada e quantos registros foram retornados.

3.7 Alteração de defesa indicada no tutorial

A alteração foi feita no arquivo C:\xampp\htdocs\www\login1.php, no Notepad++. Conforme a numeração da versão ilustrada no PDF, os caracteres // foram retirados das linhas 51 e 52, ativando addslashes(), e acrescentados às linhas 54 e 55, desativando a leitura sem tratamento. O arquivo foi salvo antes do reteste. A identificação das linhas considerou o conteúdo das atribuições, em correspondência com essa numeração.

Trecho ativo antes da alteração

```
$username = $_REQUEST['uid'];
$pass = $_REQUEST['password'];
```

Trecho após a alteração proposta no tutorial

```
$username = addslashes($_REQUEST['uid']);
$pass = addslashes($_REQUEST['password']);
// $username = $_REQUEST['uid'];
// $pass = $_REQUEST['password'];
```

Essas atribuições permanecem dentro do bloco de processamento existente, condicionado à presença de uid. A grafia correta da função é addslashes(), apesar da ocorrência de addlashes no texto explicativo do PDF. A função insere barras invertidas antes de determinados caracteres. No reteste, as entradas exemplificadas foram rejeitadas. Esse resultado é compatível com a aspa ter deixado de encerrar o literal quando o banco reconhece o escape produzido por addslashes(). A concatenação continua existindo, e o bloqueio observado convive com a consulta ainda montada por concatenação, ponto retomado na discussão da defesa.

3.8 Validação após a alteração

O reteste começou pela verificação do login com aluno_teste e Laboratorio123, e o acesso legítimo foi preservado. A sequência incluiu logout e repetição do controle negativo, com rejeição de SenhaIncorreta. Em seguida, as entradas A e B foram reenviadas separadamente no campo Usuário, com SenhaIncorreta no campo de senha. As duas tentativas de injeção foram rejeitadas.

A redação da página 24 mistura a referência aos comandos com a demonstração de acesso legítimo. A leitura conjunta com a página 25 esclarece a sequência executada: credenciais válidas permitem entrar, logout encerra a sessão e a tentativa de injeção subsequente é rejeitada. Essa distinção evita interpretar a etapa de validação como sucesso do ataque após a correção.

4 Resultados obtidos e análise

O quadro relaciona os testes aos resultados obtidos. As credenciais são fictícias e os identificadores A e B correspondem às entradas transcritas na seção 3.5.

Teste	Estado do código	Dados enviados	Resultado obtido
T1	Antes da alteração	aluno_teste / Laboratorio123	Acesso permitido.
T2	Antes da alteração	aluno_teste / SenhaIncorreta	Acesso rejeitado.
T3	Antes da alteração	Entrada A / SenhaIncorreta	Acesso indevido permitido.
T4	Antes da alteração	Entrada B / SenhaIncorreta	Acesso indevido permitido.
T5	Após addslashes()	aluno_teste / Laboratorio123	Acesso permitido.
T6	Após addslashes()	aluno_teste / SenhaIncorreta	Acesso rejeitado.
T7	Após addslashes()	Entrada A / SenhaIncorreta	Ataque rejeitado.
T8	Após addslashes()	Entrada B / SenhaIncorreta	Ataque rejeitado.

A comparação entre T3–T4 e T7–T8 mostra o efeito da alteração sobre as duas entradas. T1 e T5 registram a preservação do acesso legítimo; T2 e T6 registram a rejeição de uma senha incorreta com usuário literal.

Os testes têm funções distintas, e a conclusão depende de lê-los em conjunto. T1 e T2 formam a linha de base: com usuário literal, a aplicação aceitou a senha correta e recusou a senha incorreta. T3 e T4 isolam o efeito da entrada, porque a senha permaneceu incorreta e o acesso foi concedido. T5 e T6 mostram que a ativação de addslashes() preservou essa distinção legítima. T7 e T8 mostram que as entradas de T3 e T4 foram rejeitadas.

O conjunto sustenta que a alteração bloqueou os dois comandos exemplificados e manteve o login válido. A mesma tabela, lida com o código, sustenta uma ressalva: a consulta segue concatenada. O tutorial trata o bloqueio desses comandos como correção do exercício. O manual do PHP retira de addslashes() o papel de defesa específica de SQL. A OWASP desloca a proteção para parâmetros vinculados. As três posições se reconciliam quando o alcance de cada uma fica explícito. O tutorial descreve com fidelidade o que T7 e T8 mostraram neste ambiente. A extensão desse bloqueio a qualquer outra entrada encontra o limite indicado pelo manual e pela OWASP, desenvolvido na seção 5. A conta selecionada em T3 e T4 e a existência de outras consultas construídas com entrada externa permanecem sem evidência nesta execução.

5 Discussão técnica da defesa

A correção com addslashes() atende ao procedimento do material didático, mas não elimina de forma geral o risco de injeção SQL. O manual do PHP alerta que a função não deve ser adotada como proteção específica para consultas SQL. O efeito do escape depende do contexto e das configurações do banco. Além disso, aplicar essa função à senha antes de md5() pode alterar o valor calculado para senhas com caracteres especiais se o cadastro não realizar transformação compatível.

A defesa recomendada é separar a instrução SQL de seus valores por meio de consultas preparadas e parâmetros vinculados. O complemento abaixo usa somente colunas confirmadas no tutorial. Ele substitui a montagem e execução vulneráveis; pressupõe uma conexão PDO válida em $pdo, configurada para instruções preparadas nativas, e valores de entrada lidos como strings sem aplicação prévia de addslashes().

```
$stmt = $pdo->prepare(
'SELECT username FROM users '
. 'WHERE username = :usuario AND password = :senha LIMIT 1'
);
$stmt->execute([
':usuario' => $username,
':senha' => md5($pass),
]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);
// Só iniciar a sessão autenticada se $user !== false.
```

Nesse modelo, as entradas A e B são valores a comparar com username e não modificam a sintaxe da consulta. O código mantém MD5 apenas para compatibilidade conceitual com o exercício. A substituição do armazenamento de senhas por password_hash() e sua verificação por password_verify() exigiria uma mudança separada no cadastro e nos registros existentes. Esse complemento apresenta uma evolução da defesa, além da alteração didática solicitada pelo PDF.

Outras medidas complementares são limitar as permissões da conta usada pelo banco e evitar expor detalhes das consultas em mensagens de erro. A rejeição de dois comandos específicos, isoladamente, não demonstra ausência de outras vulnerabilidades; a revisão deve examinar todos os pontos de construção de consultas com entradas externas.

6 Conclusão

O relatório registra as etapas executadas e relaciona cada uma aos critérios do enunciado. O teste demonstrou que a concatenação de dados ao SQL contornou a autenticação por meio de uma condição sempre verdadeira e de um comentário que desativou a comparação de senha. O ajuste com addslashes() bloqueou as entradas exemplificadas neste ambiente, e a consulta parametrizada trata a causa estrutural da vulnerabilidade.

A validação incluiu a preservação do acesso legítimo e a rejeição das entradas de ataque. A comparação entre os testes explica o efeito da defesa e reforça a importância de verificar o funcionamento da autenticação após alterações no código.

As evidências sustentam o bloqueio das entradas A e B após a alteração e a permanência do acesso legítimo. A correção da causa está na separação entre dados e instrução, porque o código com addslashes() continua concatenando a entrada à consulta. Essa é a ponderação entre o resultado obtido no laboratório, a defesa pedida pelo tutorial e a posição do manual do PHP e da OWASP.

Referências

SENAI. Segurança Cibernética. Pós-graduação em Indústria 4.0. [S. l.]: SENAI, [s. d.]. Tutorial fornecido com a atividade, 25 p. Arquivo: SENAI-PosI4.0-SegurancaCibernetica-Tutorial (1).pdf.

OWASP. SQL Injection Prevention Cheat Sheet. [S. d.]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. Acesso em: 22 set. 2026.

PHP. Injeção de SQL. Manual do PHP. [S. d.a]. Disponível em: https://www.php.net/manual/pt_BR/security.database.sql-injection.php. Acesso em: 22 set. 2026.

PHP. PDO::prepare. Manual do PHP. [S. d.b]. Disponível em: https://www.php.net/manual/pt_BR/pdo.prepare.php. Acesso em: 22 set. 2026.